

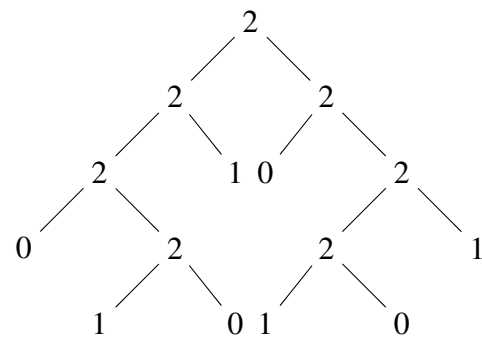
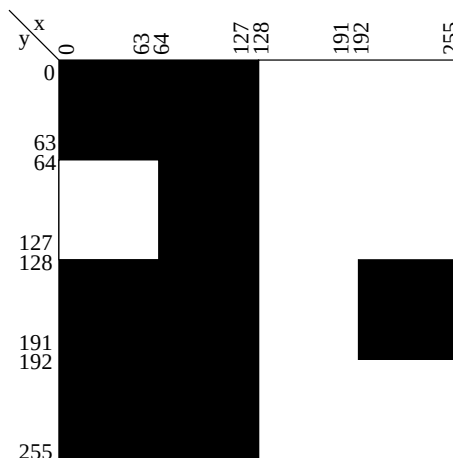
Programmation – Arbres binaires

On désire conserver l'état d'une fenêtre graphique (256×256) sous la forme d'un arbre binaire d'objets Node entiers. Chaque pixel de la fenêtre est soit éteint ($state=0$), soit allumé ($state=1$). On associe à une région quelconque de la fenêtre un des trois états suivants :

- 0 si tous les pixels de la région sont éteints,
- 1 si tous les pixels de la région sont allumés,
- 2 sinon.

Pour représenter la fenêtre dans un arbre binaire, on procède par découpages successifs de la fenêtre en deux régions égales, d'abord horizontalement, puis verticalement, puis horizontalement, ... jusqu'à obtention de régions (carrées ou rectangulaires) dont l'état est 0 ou 1. Dans l'arbre binaire associé, les nœuds de niveau pair correspondent à une région carrée et ceux de niveau impair à une région rectangulaire.

Exemple (les pixels éteints sont en noir, les pixels allumés sont en blanc) :



fournit l'arbre associé à un dessin composé par allumage des rectangles ayant pour coordonnées $[0, 64, 63, 127]$, $[128, 0, 255, 127]$, $[128, 128, 191, 191]$ et $[128, 192, 255, 255]$.

La classe `TpArbre` permet d'obtenir une configuration écran comportant cinq fenêtres graphiques de taille 256×256 , correspondant à cinq images que le programme peut traiter.

Les images sont représentées par la classe

```
public class Image extends AbstractImage
```

La classe `AbstractImage` possède les méthodes suivantes qui peuvent être testées à l'aide de la classe `TpArbre` (**this**, *image1* et *image2* désignent des images distinctes) :

Le but du TP est d'écrire les méthodes de la classe `Image`.

```
1 public abstract class AbstractImage extends BinaryTree<Node> {
2
3     public AbstractImage() {
4         super();
5     }
6
7     /**
8      * Crée this à partir d'un fichier texte (cf a1.arb, ...) et
9      * l'affiche dans une fenêtre. Chaque ligne du fichier est de la
10     * forme (e x1 y1 x2 y2) et indique si on souhaite éteindre (e=0)
11     * ou allumer (e=1) la région rectangulaire de coordonnées
12     * x1, y1, x2, y2. Le fichier se termine par un e de valeur -1.
13     */
14     public void constructTreeFromFile() { ... }
15
16     /**
17     * Sauvegarde, dans un fichier texte, les feuilles de this selon
18     * un format conforme aux fichiers manipulés par la commande
19     * constructTreeFromFile.
20     *
21     * @pre          !image.isEmpty()
22     */
23     public void saveImage() { ... }
24
25     /**
26     * Affiche this sous forme d'arbre dans une fenêtre externe.
27     *
28     * @pre          !this.isEmpty()
29     */
30     public void plotTree() { ... }
31
32     /**
33     * @pre          !image.isEmpty()
34     * @return       hauteur de this
35     */
36     public int height() { ... }
37
38     /**
39     * @pre          !image.isEmpty()
40     * @return       nombre de nœuds de this
41     */
42     public int numberOfNodes() { ... }
43
44     /**
```

```
45     * @param x      abscisse du point
46     * @param y      ordonnée du point
47     * @pre          !image.isEmpty()
48     * @return       true, si le point (x, y) est allumé dans this,
49     *               false sinon
50     */
51     abstract public boolean isPixelOn(int x, int y);
52
53     /**
54     * this devient identique à image.
55     *
56     * @pre          !image.isEmpty()
57     */
58     abstract public void affect(AbstractImage image);
59
60     /**
61     * this devient inverse vidéo de this, pixel par pixel.
62     *
63     * @pre          !image.isEmpty()
64     */
65     abstract public void videoInverse();
66
67     /**
68     * this devient rotation de image à 180 degrés
69     *
70     * @param image    image pour rotation
71     * @pre          !image.isEmpty()
72     */
73     abstract public void rotate180(AbstractImage image);
74
75     // fonction non demandée
76     abstract public void rotate90(AbstractImage image);
77
78     /**
79     * this devient image miroir vertical de image.
80     *
81     * @param image    image à agrandir
82     * @pre          !image.isEmpty()
83     */
84     abstract public void mirrorImageV(AbstractImage image);
85
86     /**
87     * this devient image miroir horizontal de image.
88     *
89     * @param image    image à agrandir
```

```
90     * @pre          !image.isEmpty()
91     */
92     abstract public void mirrorImageH(AbstractImage image);
93
94     /**
95     * this devient quart supérieur gauche de image.
96     *
97     * @param image  image à agrandir
98     * @pre          !image.isEmpty()
99     */
100    abstract public void zoomIn(AbstractImage image);
101
102    /**
103    * Le quart supérieur gauche de this devient image, le reste de
104    * this devient éteint.
105    *
106    * @param image  image à réduire
107    * @pre          !image.isEmpty()
108    */
109    abstract public void zoomOut(AbstractImage image);
110
111    /**
112    * this devient l'intersection de image1 et image2
113    * au sens des pixels allumés.
114    *
115    * @param image1 première image
116    * @param image2 deuxième image
117    * @pre          !image1.isEmpty() && !image2.isEmpty()
118    */
119    abstract public void intersection(AbstractImage image1,
120                                     AbstractImage image2);
121
122    /**
123    * this devient l'union de image1 et image2
124    * au sens des pixels allumés.
125    *
126    * @param image1 première image
127    * @param image2 deuxième image
128    * @pre          !image1.isEmpty() && !image2.isEmpty()
129    */
130    abstract public void union(AbstractImage image1,
131                              AbstractImage image2);
132
133    /**
134    * Fonction faite sans utiliser la commande isPixelOn.
```

```
135      *
136      * @pre      !this.isEmpty()
137      * @return    true, si tous les points de la forme (x, x)
138      *            (avec 0 <= x <= 255) sont, ou non, allumés
139      *            dans this, false sinon
140      */
141  abstract public boolean testDiagonal();
142
143  // fonction non demandée
144  abstract public boolean sameLeaf();
145
146  /**
147   * @param image  seconde image
148   * @pre          !this.isEmpty() && !image.isEmpty()
149   * @return       true si this est inclus dans image au sens
150   *               des pixels allumés, false sinon
151   */
152  abstract public boolean isIncludedIn(AbstractImage image);
153 }
```

```
1 public final class Node {
2
3     public final int state;
4     private static final Node ZERO = new Node(0);
5     private static final Node ONE = new Node(1);
6     private static final Node TWO = new Node(2);
7
8     /**
9      * Constructeur <b>privé</b> pour créer les objets.
10     * @param nodeValue  valeur du noeud
11     */
12     private Node(int nodeValue) { state = nodeValue; }
13
14     /**
15     * Usine à objets.
16     * @param nodeValue  valeur du noeud
17     * @return noeud avec la valeur x
18     */
19     public static Node valueOf(int nodeValue) {
20         if (nodeValue == 0) { return ZERO; }
21         else if (nodeValue == 1) { return ONE; }
22         else if (nodeValue == 2) { return TWO; }
23         return null;
24     }
25
26     @Override
27     public boolean equals(Object o) {
28         // pas besoin de tester le contenu
29         return this == o;
30     }
31
32     @Override
33     public String toString() { return "state = " + state; }
34 }
```